BACK TO METAL

WHY THIS BOOK EXISTS

YOU ARE ALLOWED TO RUN YOUR OWN COMPUTERS

A generation of engineers has now been trained to believe that owning a server is a kind of professional failure. It is not. It is a trade, and like every trade it has terms: you take on hardware, capacity and a pager, and in exchange you stop paying a margin on every byte you move and every hour you idle. For a great many companies that trade is worth making, and the arithmetic is not close.

What has been missing is not the argument. It is the **method**. Repatriation is written about as a decision and executed as a crisis, which is why so many attempts stall six months in, with half the estate moved, two platforms to run and nobody able to say whether it is going well. This book is the other thing: 74 Moves, each one a single job with a stated cutover, a stated risk and a rollback that works.

It is honest about the parts that do not pay. Several Moves here end by telling you to keep paying somebody else, because a CDN, a DDoS scrubbing centre and outbound email are three things you will not beat on your own, and a book that pretends otherwise is selling something. The point was never to own everything. It was to own the parts where ownership is cheaper, faster and yours.

Which is why the whole thing is **open source**. Every Move, the typesetter that turns them into this book, and the site that publishes them are yours: the text under Creative Commons, the software under the MIT licence, all of it at github.com/hackerbay/back-to-metal. Correct it, extend it, add the services your own estate actually runs. Infrastructure knowledge this practical should not sit behind a consulting invoice.

BACK TO METAL

Leaving the cloud with Kubernetes

Nawaz Dhandala

BY THE MAKERS OF ONEUPTIME · ONEUPTIME.COM

HACKERBAY · HACKERBAY.IO

BACK TO METAL

Leaving the cloud with Kubernetes

Published by HackerBay, HackerBay.io

First edition, 2026. Version 0.1.0.

Every runbook in this book changes production infrastructure, and several of them move data that cannot be recreated. Read the Rollback section before the first step of any Move, take a backup you have restored at least once, and rehearse against a copy. Prices, service names and product behaviour are those the author observed while writing and will drift; the figures in each Move are illustrative of the shape of a saving, not a quotation. Nothing here is legal, financial, security or compliance advice. You remain responsible for your own systems, your own data and your own obligations to the people whose data it is.

Disclosure: the author founded OneUptime, which this book recommends for alerting, on-call, incidents and status pages. It is recommended because it is Apache-2.0 licensed, self-hostable, and does in one place what that Part would otherwise ask you to assemble from four separate tools. The alternatives named beside it are real ones, and a reader who prefers them loses nothing else in this book.

Set in Archivo, drawn by Omnibus-Type, and JetBrains Mono, drawn by Philipp Nurullin and Konstantin Bulenkov. Both are licensed under the SIL Open Font License.

backtometal.hackerbay.io

MANY MOVES, NOT ONE MIGRATION

The reason cloud repatriations fail is almost never that the technology did not work. It is that they were attempted as one decision, executed as one project, and abandoned somewhere in the middle with two platforms running and nobody able to say whether it was going well.

This book takes the opposite shape. It is 74 Moves. Each is one job with a stated cutover, a stated risk and a rollback that has been thought about. Each can be done on a Tuesday and undone on a Wednesday. You can stop after any of them and still be in a coherent place, which is the only property that matters in a project measured in quarters.

They are ordered so that a reader who starts at Move 01 and works forward has, by construction, met every prerequisite of the Move in front of them. The dependency at the foot of each page always points backwards. That is not a stylistic choice; the build refuses to compile a book where it does not hold.

Every Move carries all three clouds. The job is the same whether you are leaving AWS, Google Cloud or Azure — what differs is the extraction, so each Move opens with three lines naming the real service on each provider and the one thing that is different there. Writing three whole variants of every runbook would have tripled the book to say the same thing three times.

Three of the Moves conclude that you should keep paying somebody else. A global content network, scrubbing capacity at the edge and outbound email are businesses other people run better than you will, and each is cheap next to what it replaces. The aim was never to own everything.

Work in any order the dependencies allow. Rehearse everything. And read the Rollback section before the first step of every Move, including the ones that look like nothing.

EVERY MOVE, WITHOUT EXCEPTION

HOW A MOVE WORKS

1. **THE RING**
   The cutover: how many minutes of user-visible downtime this Move costs, against a 60-minute dial. An empty ring means none, and most rings are empty.
2. **THE BARS**
   Risk, meaning blast radius if it goes wrong rather than how fiddly it is. Three bars and you do not run it alone.
3. **THE PANEL**
   Everything that must already be true before step one. Hardware, software with versions, and the access you will be asked for.
4. **WHY THIS WORKS**
   What the managed service actually was, and why the replacement holds. Read it once; you will not need it again.
5. **THE STRIP**
   What it cost, what it costs now, the saving, the downtime and the person-days. Illustrative of a shape, not a quotation for your account.
6. **THE ROLLBACK**
   On the facing page, at the foot, in colour. It names the point of no return. Read it before step one, not after step five.

THE REFERENCE BUILD

Every Move in this book is written against one cluster, so that a runbook can name a real thing rather than a category: 2 sites, 6 nodes each, 64 cores and 512 GB a node, 25 GbE to the top of the rack. Two sites because one site is not a platform, and 6 nodes because that is where losing one stops hurting. Scale the numbers; do not scale away the redundancy.

COMPUTE

- [ ] Six 1U or 2U dual-socket servers per site
- [ ] 64 physical cores a node, current-generation EPYC or Xeon
- [ ] 512 GB ECC RAM a node, all channels populated
- [ ] Redundant hot-swap power supplies, both fed
- [ ] BMC on every node - Redfish, not just a web console
- [ ] Two spare nodes on the shelf, racked and burned in

STORAGE

- [ ] Four NVMe drives a node, mixed-use endurance, 3.84 TB each
- [ ] Enterprise drives with power-loss protection - not consumer SSDs
- [ ] A separate boot pair, mirrored
- [ ] One spare drive of each size, on site, unopened
- [ ] A backup target in a third location you do not control

NETWORK

- [ ] Two 25 GbE ToR switches a site, MLAG or stacked
- [ ] Two uplinks a node, LACP to separate switches
- [ ] A separate 1 GbE out-of-band management switch
- [ ] Your own address space, or a /24 from the facility
- [ ] Two transit providers, or one transit and one IX
- [ ] A serial console server, reachable when the cluster is not

SITE AND POWER

- [ ] Two facilities far enough apart to fail separately
- [ ] A- and B-side power, on separate PDUs
- [ ] Remote hands with a response time in the contract
- [ ] Cross-connects ordered and tested before you need them
- [ ] Physical access for at least two named people
- [ ] A written escalation path with a phone number on it

THE SOFTWARE STACK

- [ ] Talos Linux on every node - no shell, no package manager, no drift
- [ ] Kubernetes, on a version at least one minor behind the newest
- [ ] Cilium for CNI, with kube-proxy replacement
- [ ] Rook, and the Ceph cluster it manages
- [ ] cert-manager, external-dns, and Envoy Gateway
- [ ] Argo CD, with the cluster state in Git
- [ ] Prometheus and Loki for the stores, OneUptime for what wakes someone
- [ ] Velero, and a restore you have actually performed

ACCESS AND IDENTITY

- [ ] An OIDC provider that is not the cluster
- [ ] Break-glass credentials in a safe, on paper
- [ ] A bastion or a mesh - not a public API server
- [ ] Hardware tokens for anyone who can delete a cluster
- [ ] An audit log shipped somewhere the cluster cannot reach
- [ ] A status page that is not hosted on the thing it reports on
- [ ] The domain registrar account locked, with recovery codes printed

ON THE LAPTOP

- [ ] kubectl
- [ ] helm
- [ ] talosctl
- [ ] k9s
- [ ] argocd
- [ ] velero
- [ ] psql and pg_dump
- [ ] mysql client
- [ ] redis-cli
- [ ] rclone
- [ ] aws CLI, still logged in
- [ ] dig and mtr
- [ ] openssl
- [ ] ipmitool
- [ ] a serial cable
- [ ] the runbook, printed

THE ON-RAMP

Almost nobody should sign a facility contract before they have run this stack once. A cluster of 3 refurbished machines with 12 cores and 64 GB each, on a managed switch under a desk, will run every Move in Parts II and III unchanged and most of Part IV besides. It costs about $1,840 once and about $30 a month in electricity, it saves nothing whatsoever, and it is the cheapest way to find out whether the rest of this book is for you.

WHY THREE, AND WHY SECOND-HAND

Three because a control plane needs three members to lose one and keep going, and everything you will learn about quorum, fencing and upgrades needs somewhere to go wrong. Second-hand because a machine two generations old runs Kubernetes exactly as well as a new one at roughly a tenth of the price, and because you want to be willing to break it.

WHAT IT CANNOT TEACH YOU

Every word of Part I. A homelab has one power feed, one switch, no cross-connect, no remote hands, no second site and nobody to escalate to at three in the morning. It cannot show you what a colocation contract is for, or what happens when the A feed goes away and the B feed was never tested. The building is a different discipline, which is why it has a Part of its own.

THE ONE PART TO BUY NEW

The drives. A consumer SSD without power-loss protection will corrupt etcd the first time the power flickers, and it will do it in a way that looks like a Kubernetes bug for a day and a half. Buy small enterprise NVMe with the protection, second-hand if you like, and put the money you saved on the chassis into the disks.

WHAT IT GENUINELY PROVES

The whole software estate. Talos, the control plane, Cilium with kube-proxy replaced, Rook and Ceph, the registry, secrets, the observability stack, and a Postgres cutover rehearsed end to end against a copy. If a Move works here it will work on the metal; the difference is scale, not shape.

WHEN TO STOP USING IT

Never. The on-ramp becomes the non-production cluster the rest of the book keeps asking for — the one you restore etcd onto, rehearse an upgrade on, and point a load generator at. Every estate needs a cluster it is allowed to destroy, and this is the cheapest one you will ever own.

IF THREE MACHINES AT HOME IS NOT POSSIBLE

Rent three of the cheapest dedicated hosts you can find, by the month, and run exactly the same experiment. It costs more than electricity and less than being wrong, it gives you real addresses and a real network, and you can hand it all back at the end of the month. What it will not give you is a machine you can physically pull a disk out of.

THE WHOLE SHOPPING LIST

- [ ] Three refurbished mini PCs or small-form-factor desktops
- [ ] 64 GB in each - memory is what runs out first, not cores
- [ ] Two NVMe drives a node: one to boot, one for data
- [ ] A managed switch with VLANs and a spare port for out-of-band
- [ ] A small UPS, mostly to survive the brownouts that corrupt etcd
- [ ] A label printer, because you will regret not having one
- [ ] A separate router or firewall you are willing to break
- [ ] A domain you own, for real certificates rather than self-signed ones

TEN RULES FOR LEAVING THE CLOUD

1 **Move the cheapest thing first.** Not the most expensive. The first Move exists to prove the platform and the team, and it should be something whose failure is a shrug. You are buying evidence, and evidence is worth more early than savings are.

2 **Never run two platforms longer than you planned to.** The cost of a repatriation is not the hardware. It is the months in which both estates are live, both are on call, and every change has to be made twice. Pick the overlap window before you start, write it down, and treat overrunning it as the incident it is.

3 **Data moves last and leaves last.** Every stateless thing can be cut over and cut back in minutes. State cannot. Move compute first, prove it against the cloud database across the link, and only then move the database - by which point you will have learned the things you would otherwise have learned during a data migration.

4 **Keep the bill until the invoice proves you can stop.** A resource is not decommissioned when nothing points at it. It is decommissioned when a full month has passed, the line has gone from the bill, and nobody has filed a ticket. Turning things off too early is how a migration becomes an outage a fortnight later.

5 **Buy the second of everything before you need the first.** Two switches, two power feeds, two transit providers, two people who can get into the building. Every single point of failure you accept at the start becomes an incident you will attend in person, at night, in a year.

6 **Own your addresses.** An IP range you rent from a provider is a migration you have to do again. Your own address space and your own ASN cost a few hundred pounds and take a few weeks, and they are the difference between changing suppliers and changing everything.

7 **Automate the rebuild, not the build.** Anyone can install a cluster once. What matters is whether you can lose a site and rebuild it from a repository and a backup, with no hands on the original. If the answer involves remembering something, you do not have a platform yet.

8 **Count the salary.** A saving that costs one and a half engineers is a real saving only if you write the one and a half engineers into the comparison. Most repatriations still win by a wide margin once you do. The ones that do not, you want to know about in month one rather than month ten.

9 **Keep paying for the three things you will not beat.** A global CDN, DDoS scrubbing at the edge, and outbound email deliverability. Each is a business somebody else is already running better than you will, and each is cheap next to what it replaces. Ownership is a means, not a principle.

10 **The migration is finished when the account is closed.** Not when the traffic moves. An estate with a dormant cloud account still has the cost, the credentials, the compliance surface and the temptation. The last Move in this book is closing the account, and until it is done the project is still open.

BEFORE YOU TOUCH ANYTHING

Every Move in this book changes something that is running. Most are reversible for a stated window, a few are reversible only until a particular step, and three are not reversible at all. The difference between a migration and an incident is almost never the technique - it is whether somebody worked out the way back before they started, and whether anybody had ever tested it.

READ THE ROLLBACK FIRST

Not the runbook. The Rollback section of every Move names the point of no return: the step after which the old system can no longer serve traffic or no longer holds current data. Read it, decide whether you are willing to reach that step today, and only then start at step one.

KEEP THE OLD THING RUNNING, AND KEEP PAYING FOR IT

The instinct after a successful cutover is to delete the source and book the saving. Do not. The Reversible field on each Move is how long the old system must stay warm, powered and receiving its own backups. That overlap is the entire cost of being wrong, and it is cheap.

WRITE DOWN THE STATE YOU CANNOT RECREATE

Before Part II, list every store whose contents exist nowhere else: the primary database, the uploads bucket, the secrets, the certificate private keys, the TOTP seeds for the accounts that own the domain. Everything else is rebuildable from a repository. That list is short, and it is the only part of the estate where a mistake is permanent.

TWO PEOPLE, ONE KEYBOARD

Every High-risk Move in this book assumes a second person who is not typing, is reading the runbook aloud, and has the authority to stop. It is the cheapest safety control in infrastructure and the first one teams drop when they are behind schedule.

A BACKUP NOBODY RESTORED IS NOT A BACKUP

It is a file. Before any Move in Part II, restore the backup you are relying on into a scratch environment and count the rows against production. The number of organisations that discover their backups were empty during the incident that needed them is not small, and every one of them had a green dashboard.

CUT OVER WHEN YOU CAN UNDO IT, NOT WHEN YOU ARE CONFIDENT

Confidence is not evidence. The right time to move traffic is when the rollback has been rehearsed end to end, on the same day of the week, with the same people awake. A Tuesday morning with the team at their desks beats a Saturday night with one person and a laptop, every time.

VERIFY BY COUNTING, NOT BY LOOKING

A migration is finished when the destination answers the same questions as the source with the same answers. Row counts, checksums, a replayed hour of real traffic compared response by response. A page that loads is not evidence, and neither is an application that starts.

STOP WHEN YOU ARE SURPRISED

Not when something breaks - when something is merely unexpected. A row count that is off by eleven, a certificate with the wrong issuer, a replica that caught up faster than it should have. Surprise means the model in your head and the system in front of you have diverged, and continuing is guessing.

These are operational practices, not a warranty. The runbooks in this book were written against the services and versions named in each Move and will drift as those change. Rehearse every Move against a copy of your own estate before you run it against the real one, and treat any figure here as the shape of an answer rather than a quotation for your account.

THE ARITHMETIC, WITH THE SALARY IN IT

WHAT IT ACTUALLY COSTS

The comparison that gets a repatriation approved and then regretted is the one that counts the hardware and forgets the people. This one counts both, at list price and with no commitment discount on either side, for the reference build opposite. Substitute your own effective rate; the shape does not change.

ON AWS, COMPUTE ALONE

1,536 vCPU of current-generation general purpose instances, on demand, is **$56,513** a month before a single gigabyte of storage, a load balancer or a byte of egress. Egress is the line that ends most arguments: 100 TB a month is $9,216, every month, for the privilege of your own traffic leaving.

OWNED, IN TWO FACILITIES

$12,087 of infrastructure - metal amortised over five years, power, racks, transit, cross-connects and remote hands - plus $26,250 of additional salaried time. Total **$38,337** a month for the same 1,536 vCPU, with the redundancy the cloud figure does not include.

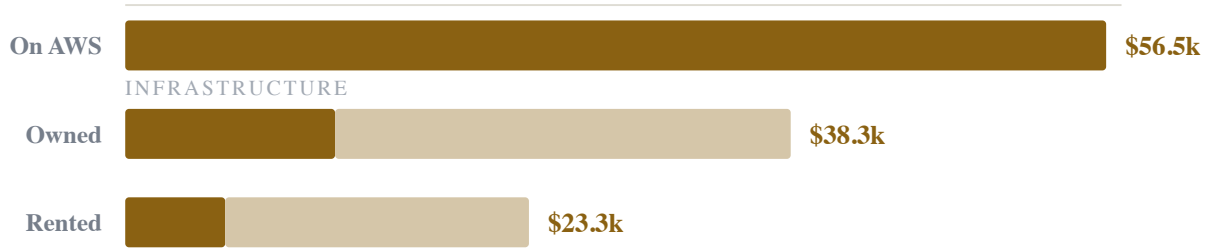
RENTED BY THE MONTH

The same class of machine from a dedicated-host provider is $5,760, and removes the racking, the spares and half a person: **$23,260** all in. Most of the saving, none of the loading dock, and a supplier you can still leave in thirty days.

WHAT THE DIFFERENCE BUYS

Roughly $18,175 a month, or $218,102 a year, against a capital outlay of $288,000 that pays for itself in about 16 months. Every Move in this book states its own version of this table, and every one of them is a fraction of this.

THE SAME 1,536 VCPU, THREE WAYS, PER MONTH



Every AWS figure is a public list price for us-east-1 observed while writing, before any Savings Plan, private pricing agreement or credit. Every hardware figure is a mid-market street price for the specification opposite. Both will drift. The comparison is a method you can repeat against your own invoice, not a quotation - and if your effective rate is half of list, halve the left-hand column and read the conclusion again.

WHAT REPLACES WHAT

Every Move names the real service on all three clouds and the one thing that differs on each, so this table is the map rather than the content. Find the row you are paying for and then read the Move. The right-hand column is the point of the whole book, and where it says keep paying, that is a conclusion rather than a gap.

AWS	GOOGLE CLOUD	AZURE	WHAT YOU RUN INSTEAD
EC2	Compute Engine	Virtual Machines	Bare metal you own, or dedicated hosts by the month
EKS	GKE	AKS	Talos Linux, and Kubernetes you run yourself
Fargate	Cloud Run	Container Apps	Deployments and Jobs on your own nodes
Auto Scaling groups	Managed instance groups	Scale sets	Fixed capacity, plus a spare shelf and a burst account
AMI	Machine images	Managed images	A Talos machine config, applied over the network
RDS for PostgreSQL	Cloud SQL for PostgreSQL	Database for PostgreSQL	PostgreSQL under CloudNativePG
Aurora	AlloyDB	Cosmos DB for PostgreSQL	PostgreSQL with read replicas - the storage layer does not come with you
RDS for MySQL	Cloud SQL for MySQL	Database for MySQL	MySQL or MariaDB under an operator
ElastiCache	Memorystore	Azure Cache for Redis	Valkey or Redis, on local NVMe
S3	Cloud Storage	Blob Storage	Ceph RGW under Rook - and S3 kept for cold archival
EBS	Persistent Disk	Managed Disks	Ceph RBD under Rook, with local NVMe underneath it
EFS	Filestore	Azure Files	CephFS under Rook, or a filesystem the application stops needing
MSK	Pub/Sub, Managed Kafka	Event Hubs	Kafka under Strimzi
SQS and SNS	Pub/Sub	Service Bus	NATS JetStream, or a table in the database you already have
OpenSearch Service	Elasticsearch on GKE	Azure AI Search	OpenSearch, or Postgres full-text where it is enough
DynamoDB	Bigtable, Firestore	Cosmos DB	PostgreSQL - the hardest lock-in in the catalogue
Redshift	BigQuery	Synapse	ClickHouse
AWS Backup	Backup and DR	Azure Backup	Velero, pgBackRest and restic, to a third site
ALB and NLB	Cloud Load Balancing	Load Balancer	Cilium in BGP mode, behind addresses you own
CloudFront	Cloud CDN	Azure Front Door	**Keep paying - Bunny, Fastly or Cloudflare**
Route 53	Cloud DNS	Azure DNS	Two independent authoritative providers, never one

WHAT REPLACES WHAT II

Three rows here say keep paying, and they are set in red because they are the most important lines in the table. A global content network, scrubbing capacity at the edge and outbound mail deliverability are not technical problems you have not solved yet. They are businesses, and you are not in them.

AWS	GOOGLE CLOUD	AZURE	WHAT YOU RUN INSTEAD
ACM	Certificate Manager	App Service Certificates	cert-manager with ACME, plus an internal CA
AWS WAF and Shield	Cloud Armor	Azure WAF and DDoS Protection	**Keep paying for scrubbing; run the WAF rules yourself**
Global Accelerator	Premium Tier network	Front Door	Anycast from your own ASN, or the CDN you kept
NAT Gateway	Cloud NAT	NAT Gateway	A router. This is the single most absurd line on the bill
SES	No first-party equivalent	Communication Services	**Keep paying - deliverability is a reputation you cannot buy back**
ECR	Artifact Registry	Container Registry	Harbor, with a pull-through cache
CodeBuild and CodePipeline	Cloud Build	Azure Pipelines	Self-hosted runners on real NVMe, or Tekton
Secrets Manager	Secret Manager	Key Vault	External Secrets with Vault, or SOPS in Git
KMS	Cloud KMS	Key Vault	Vault Transit - the root of trust is the hard part
IAM roles for service accounts	Workload Identity	Workload Identity	SPIFFE and SPIRE
CloudWatch metrics	Cloud Monitoring	Azure Monitor	Prometheus, or VictoriaMetrics at scale
CloudWatch Logs	Cloud Logging	Log Analytics	Vector into Loki
X-Ray	Cloud Trace	Application Insights	OpenTelemetry into Tempo
Lambda	Cloud Functions	Functions	Jobs, CronJobs and KEDA
Step Functions	Workflows	Logic Apps	Argo Workflows
Cost Explorer	Cost management	Cost Management	OpenCost, and an accountant who understands depreciation
CloudWatch alarms and SNS	Cloud Monitoring alerting	Azure Monitor alerts	OneUptime - alerting, on-call rotas and escalation
Systems Manager Incident Manager	No first-party equivalent	No first-party equivalent	OneUptime - incidents, timelines and postmortems
No first-party equivalent	No first-party equivalent	No first-party equivalent	OneUptime - a status page your customers can read
Synthetic Canaries	Uptime checks	Standard Test (Preview)	OneUptime - synthetic and uptime monitoring

WORK OUT WHAT TO BUY

DONE WHEN You know what to order, and why.

What to buy. Cores, memory, disks, network cards, switches, optics and the topology they hang on — chosen against your own numbers rather than a vendor configurator, and tested before anything depends on it.

☐	01	The bill of materials	5 min	24
☐	02	Cores against clock, and the socket count	0 min	26
☐	03	Memory is what runs out first	30 min	28
☐	04	The drives that will not eat your etcd	12 min	30
☐	05	Network cards, and the two ports you actually need	12 min	32
☐	06	Leaf-spine, or two switches and a cable	0 min	34
☐	07	Optics, coding and the vendor lock	30 min	36
☐	08	Tier one, whitebox, or somebody else's three-year-old fleet	30 min	38
☐	09	Burn-in before anything trusts it	0 min	40
☐	10	What to keep on the shelf	0 min	42

FIND SOMEWHERE TO PUT IT

DONE WHEN The machines are racked, powered and reachable.

Where the machines live, who owns the room, and what you are signing. Nothing here is software, and getting it wrong is the only mistake in this book you cannot fix with a deploy.

- [ ] 11 Colo, your own room, or rented metal — 12 min — 46
- [ ] 12 Reading the power clause — 5 min — 48
- [ ] 13 The cross-connect price list — 0 min — 50
- [ ] 14 Two feeds, one of which you have tested — 5 min — 52
- [ ] 15 Cooling, and the inlet temperature you are allowed — 0 min — 54
- [ ] 16 The first rack — 0 min — 56
- [ ] 17 Who gets in at 03:00 — 30 min — 58
- [ ] 18 The loading dock and the customs form — 20 min — 60
- [ ] 19 Certified destruction — 3 min — 62

BUILD THE CLUSTER

DONE WHEN A cluster that can take production traffic, and has never seen any.

Everything between a racked machine and a cluster that can take production traffic. Nothing here moves a workload; all of it decides how well the rest of the book goes.

- [ ] 20 Talos, and the machine config — 20 min — 66
- [ ] 21 Colo or rented metal, and the second site you are not building — 0 min — 68
- [ ] 22 The address plan, and the three clocks you start today — 0 min — 70
- [ ] 23 Netboot and node identity — 0 min — 72
- [ ] 24 The BMC, the management VLAN and the way in at 03:00 — 0 min — 74
- [ ] 25 Talos, or Flatcar when Talos will not do — 3 min — 76
- [ ] 26 Netboot, node identity and failure-domain labels — 3 min — 78
- [ ] 27 Three control-plane nodes, the API VIP and the cluster CA — 0 min — 80
- [ ] 28 The etcd restore drill — 0 min — 82
- [ ] 29 Cilium, with kube-proxy off — 0 min — 84
- [ ] 30 BGP to the top-of-rack, and the MTU that breaks five per cent — 12 min — 86

MAKE IT FIT TO RUN PRODUCTION

DONE WHEN Identity, secrets, images, policy and dashboards, before any data arrives.

What developers touch. Get this wrong and the migration succeeds while everybody who has to use it quietly hates you.

☐	31	Workload identity without the cloud's	0 min	90
☐	32	Secrets: External Secrets as the dated bridge	12 min	92
☐	33	The mirror, then Harbor as the registry of record	0 min	94
☐	34	cosign, Trivy and an admission policy	0 min	96
☐	35	Builds onto your own metal	0 min	98
☐	36	Argo CD, and the two infrastructure-as-code estates	5 min	100
☐	37	East-west: security groups into NetworkPolicy	12 min	102
☐	38	The first stateless service, end to end	5 min	104
☐	39	The root of trust after managed KMS	20 min	106
☐	40	The serverless estate in four piles	0 min	108
☐	41	Orchestrated workflows: the dated exception	20 min	110

MOVE THE DATA

DONE WHEN Your state is on your disks, and the old copy is still warm.

The part that can lose a company its data, and therefore the part with the longest overlap windows, the most rehearsal and the fewest one-way steps.

☐	42	The links you build first: colo to cloud, colo to office	5 min	114
☐	43	The Postgres pre-flight	3 min	116
☐	44	The verification gate	0 min	118
☐	45	Postgres onto your own disks	30 min	120
☐	46	Point-in-time recovery with pgBackRest	0 min	122
☐	47	Multi-zone replaced: quorum failover and fencing	20 min	124
☐	48	Aurora and AlloyDB: the exit is a stream	20 min	126
☐	49	MySQL, wherever it is managed	5 min	128
☐	50	Redis: which half of it is actually a cache	0 min	130
☐	51	The queues come home, the fan-out does not	5 min	132
☐	52	Buckets: the mirror, and the S3 API your code assumes	0 min	134

MOVE THE TRAFFIC

DONE WHEN Users are reaching your addresses, not somebody else's.

How traffic reaches you once the addresses are yours. Also the Part with the most honest advice to keep paying somebody else.

- [ ] 53 Your own ASN, a /24 and the IPv6 you already have — 0 min — 138
- [ ] 54 Transit: two upstreams and the 95th percentile — 12 min — 140
- [ ] 55 Egress off the managed NAT, and the allowlists you renegotiate — 20 min — 142
- [ ] 56 The first VIP: L2, and failover IPs when nobody will peer — 0 min — 144
- [ ] 57 VIPs over BGP: ECMP, BFD and the rehash — 20 min — 146
- [ ] 58 TLS off the managed certificate service — 0 min — 148
- [ ] 59 Envoy Gateway, after ingress-nginx — 30 min — 150
- [ ] 60 Balancer rules and nginx annotations into HTTPRoute — 0 min — 152
- [ ] 61 Edge authentication after the balancer did it — 30 min — 154
- [ ] 62 The shadow edge: two gateways and a weighted record — 12 min — 156
- [ ] 63 Go-live: the soak, the freeze, and how you abort — 20 min — 158

RUN IT, AND CLOSE THE ACCOUNT

DONE WHEN You can carry it at 03:00, and the cloud bill is zero.

Running it once it is yours. Every repatriation article stops before this Part, and every repatriation that regrets itself failed inside it.

☐	64	The upgrade calendar and the control-plane upgrade	30 min	162
☐	65	The fleet roll with no thirteenth machine	0 min	164
☐	66	The 03:00 disk	0 min	166
☐	67	Spares, RMA and which rack unit	30 min	168
☐	68	Capacity planning, and the cloud footprint you keep	0 min	170
☐	69	Velero, the off-site copy and the rebuild drill	0 min	172
☐	70	The rota	20 min	174
☐	71	Alerts that survive the cluster, and the pager you rent	20 min	176
☐	72	The patch SLA after NVD stopped	3 min	178
☐	73	Audit logs after the provider's	0 min	180
☐	74	The auditor's checklist, and the contracts nobody read	0 min	182

IRON

Servers, disks, switches and optics, chosen against your numbers and burned in before anything trusts them.

SITE

The facility, the contract and the power. The Part where a mistake costs a lorry rather than a deploy.

CLUSTER

The OS, the control plane, the network and the disks. Nothing here moves a workload, and everything here decides how the rest goes.

PLATFORM

Builds, registries, secrets, identity and observability. What your own engineers actually touch.

DATA

State, and the way back from moving it. The longest overlaps and the most rehearsal in the book.

EDGE

Addresses, balancers, DNS and certificates - and the three things you keep paying somebody else to do.

WATCH

Upgrades, on-call, capacity and compliance. The Part that decides whether this was a good idea.

Every Move states its cutover in minutes of user-visible downtime, its risk as blast radius rather than difficulty, and how long the thing it replaced must stay warm before you turn it off. 33 of the 74 Moves in this book cost no downtime at all; the whole book, executed end to end, costs 641 minutes.

WHICH MOVE DO I NEED

Nobody opens a book like this at Move 01. They open it because something on the bill, or on the pager, has become intolerable. This is the index for the way the question actually arrives.

The bill went up forty per cent and nobody can say why

10 What to keep on the shelf
58 TLS off the managed certificate service
70 The rota
48 Aurora and AlloyDB: the exit is a stream

We are being rate-limited by a database we pay a fortune for

11 Colo, your own room, or rented metal
61 Edge authentication after the balancer did it
10 What to keep on the shelf
54 Transit: two upstreams and the 95th percentile

Something has to move this quarter and nothing may break

49 MySQL, wherever it is managed
02 Cores against clock, and the socket count
10 What to keep on the shelf
11 Colo, your own room, or rented metal

A single availability zone took us down anyway

43 The Postgres pre-flight
50 Redis: which half of it is actually a cache
59 Envoy Gateway, after ingress-nginx
57 VIPs over BGP: ECMP, BFD and the rehash

Our secrets are in four places and one of them is a spreadsheet

66 The 03:00 disk
04 The drives that will not eat your etcd
40 The serverless estate in four piles
12 Reading the power clause

The audit is in eight weeks and the evidence lives in a console

64 The upgrade calendar and the control-plane upgrade
63 Go-live: the soak, the freeze, and how you abort
33 The mirror, then Harbor as the registry of record
02 Cores against clock, and the socket count

Egress is now the largest line on the invoice

06 Leaf-spine, or two switches and a cable
17 Who gets in at 03:00
44 The verification gate
46 Point-in-time recovery with pgBackRest

The finance director has asked what happens if we just left

04 The drives that will not eat your etcd
64 The upgrade calendar and the control-plane upgrade
74 The auditor's checklist, and the contracts nobody read
02 Cores against clock, and the socket count

We cannot get a straight answer about an outage we did not cause

12 Reading the power clause
15 Cooling, and the inlet temperature you are allowed
33 The mirror, then Harbor as the registry of record
54 Transit: two upstreams and the 95th percentile

The container registry is throttling our deploys

60 Balancer rules and nginx annotations into HTTPRoute
70 The rota
11 Colo, your own room, or rented metal
67 Spares, RMA and which rack unit

We have no idea what would happen if the cluster were deleted

62 The shadow edge: two gateways and a weighted record
03 Memory is what runs out first
30 BGP to the top-of-rack, and the MTU that breaks five per cent
15 Cooling, and the inlet temperature you are allowed

Nobody wants to be on call for hardware

48 Aurora and AlloyDB: the exit is a stream
39 The root of trust after managed KMS
19 Certified destruction
26 Netboot, node identity and failure-domain labels

WHICH MOVE DO I NEED II

The rest of the same index. Every number points at a Move, and the build refuses to compile a reference that does not.

The monitoring bill is larger than the thing it monitors

67 Spares, RMA and which rack unit
22 The address plan, and the three clocks you start today
44 The verification gate
57 VIPs over BGP: ECMP, BFD and the rehash

Latency between our services is worse than it should be

33 The mirror, then Harbor as the registry of record
26 Netboot, node identity and failure-domain labels
56 The first VIP: L2, and failover IPs when nobody will peer
28 The etcd restore drill

Half the estate is serverless and nobody knows what it costs

18 The loading dock and the customs form
64 The upgrade calendar and the control-plane upgrade
45 Postgres onto your own disks
06 Leaf-spine, or two switches and a cable

Someone has to hold an IP address we actually own

58 TLS off the managed certificate service
61 Edge authentication after the balancer did it
36 Argo CD, and the two infrastructure-as-code estates
28 The etcd restore drill

We are locked into one database we cannot price-compare

64 The upgrade calendar and the control-plane upgrade
31 Workload identity without the cloud's
42 The links you build first: colo to cloud, colo to office
52 Buckets: the mirror, and the S3 API your code assumes

We were told we cannot leave because of compliance

50 Redis: which half of it is actually a cache
29 Cilium, with kube-proxy off
41 Orchestrated workflows: the dated exception
27 Three control-plane nodes, the API VIP and the cluster CA

We want out but the CDN and the mail are non-negotiable

09 Burn-in before anything trusts it
36 Argo CD, and the two infrastructure-as-code estates
22 The address plan, and the three clocks you start today
15 Cooling, and the inlet temperature you are allowed

The migration stalled and we are now running two of everything

53 Your own ASN, a /24 and the IPv6 you already have
49 MySQL, wherever it is managed
67 Spares, RMA and which rack unit
64 The upgrade calendar and the control-plane upgrade

I

IRON

What to buy. Cores, memory, disks, network cards, switches, optics and the topology they hang on — chosen against your own numbers rather than a vendor configurator, and tested before anything depends on it.

10	4	6	119
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

02

CORES AGAINST CLOCK, AND THE SOCKET COUNT

Synthetic placeholder for the Iron Part, number 02, used only to design the page layouts at realistic length.

01

THE BILL OF MATERIALS

Synthetic placeholder for the Iron Part, number 01, used only to design the page layouts at realistic length.

03

MEMORY IS WHAT RUNS OUT FIRST

Synthetic placeholder for the Iron Part, number 03, used only to design the page layouts at realistic length.

THE BILL OF MATERIALS · CORES AGAINST CLOCK, AND THE SOCKET COUNT · MEMORY IS WHAT RUNS OUT FIRST · THE DRIVES THAT WILL NOT EAT YOUR ETCD · NETWORK CARDS, AND THE TWO PORTS YOU ACTUALLY NEED · LEAF-SPINE, OR TWO SWITCHES AND A CABLE · OPTICS, CODING AND THE VENDOR LOCK · TIER ONE, WHITEBOX, OR SOMEBODY ELSE'S THREE-YEAR-OLD FLEET · BURN-IN BEFORE ANYTHING TRUSTS IT · WHAT TO KEEP ON THE SHELF

3 LOW RISK	1 MEDIUM RISK	6 HIGH RISK

01 THE BILL OF MATERIALS

CUTOVER
5 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	**MANAGED SERVICE**	HIGH	NO

> Synthetic placeholder for the Iron Part, number 01, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,090/mo	$76/mo	98%	5 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

01 THE BILL OF MATERIALS

8 STEPS · 5 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 5 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

02 CORES AGAINST CLOCK, AND THE SOCKET COUNT

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	LOW	30 DAYS

Synthetic placeholder for the Iron Part, number 02, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,730/mo	$76/mo	98%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 01 The bill of materials

02 CORES AGAINST CLOCK, AND THE SOCKET COUNT

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

03 MEMORY IS WHAT RUNS OUT FIRST

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Iron Part, number 03, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$850/mo	$364/mo	57%	30 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 02 Cores against clock, and the socket count

03 MEMORY IS WHAT RUNS OUT FIRST

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

04 THE DRIVES THAT WILL NOT EAT YOUR ETCD

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Iron Part, number 04, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$850/mo	$460/mo	46%	12 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 02 Cores against clock, and the socket count

04 THE DRIVES THAT WILL NOT EAT YOUR ETCD

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

05 NETWORK CARDS, AND THE TWO PORTS YOU ACTUALLY NEED

CUTOVER
12 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Iron Part, number 05, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,660/mo	$484/mo	71%	12 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 04 The drives that will not eat your etcd

05 NETWORK CARDS, AND THE TWO PORTS YOU ACTUALLY NEED

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

06 LEAF-SPINE, OR TWO SWITCHES AND A CABLE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	MEDIUM	IMMEDIATELY

> Synthetic placeholder for the Iron Part, number 06, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,660/mo	$64/mo	96%	0 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 02 Cores against clock, and the socket count 04 The drives that will not eat your etcd

06 LEAF-SPINE, OR TWO SWITCHES AND A CABLE

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

07 OPTICS, CODING AND THE VENDOR LOCK

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Iron Part, number 07, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,740/mo	$148/mo	95%	30 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 04 The drives that will not eat your etcd 06 Leaf-spine, or two switches and a cable

07 OPTICS, CODING AND THE VENDOR LOCK

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

08 TIER ONE, WHITEBOX, OR SOMEBODY ELSE'S THREE-YEAR-OLD FLEET

CUTOVER
30 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Iron Part, number 08, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,550/mo	$172/mo	95%	30 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 07 Optics, coding and the vendor lock

08 TIER ONE, WHITEBOX, OR SOMEBODY ELSE'S THREE-YEAR-OLD FLEET

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

09 BURN-IN BEFORE ANYTHING TRUSTS IT

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	LOW	30 DAYS

> Synthetic placeholder for the Iron Part, number 09, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$940/mo	$460/mo	51%	0 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 07 Optics, coding and the vendor lock

09 BURN-IN BEFORE ANYTHING TRUSTS IT

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

10 WHAT TO KEEP ON THE SHELF

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
IRON	MANAGED SERVICE	LOW	7 DAYS

> Synthetic placeholder for the Iron Part, number 10, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,190/mo	$448/mo	86%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 02 Cores against clock, and the socket count 09 Burn-in before anything trusts it

10 WHAT TO KEEP ON THE SHELF

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

IRON · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

II

SITE

Where the machines live, who owns the room, and what you are signing. Nothing here is software, and getting it wrong is the only mistake in this book you cannot fix with a deploy.

9	3	7	75
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

13

THE CROSS-CONNECT PRICE LIST

Synthetic placeholder for the Site Part, number 13, used only to design the page layouts at realistic length.

18

THE LOADING DOCK AND THE CUSTOMS FORM

Synthetic placeholder for the Site Part, number 18, used only to design the page layouts at realistic length.

11

COLO, YOUR OWN ROOM, OR RENTED METAL

Synthetic placeholder for the Site Part, number 11, used only to design the page layouts at realistic length.

COLO, YOUR OWN ROOM, OR RENTED METAL · READING THE POWER CLAUSE · THE CROSS-CONNECT PRICE LIST · TWO FEEDS, ONE OF WHICH YOU HAVE TESTED · COOLING, AND THE INLET TEMPERATURE YOU ARE ALLOWED · THE FIRST RACK · WHO GETS IN AT 03:00 · THE LOADING DOCK AND THE CUSTOMS FORM · CERTIFIED DESTRUCTION

1 LOW RISK	1 MEDIUM RISK	7 HIGH RISK

11 COLO, YOUR OWN ROOM, OR RENTED METAL

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Site Part, number 11, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,730/mo	$388/mo	90%	12 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 03 Memory is what runs out first 05 Network cards, and the two ports you actually need 06 Leaf-spine, or two switches and a cable 07 Optics, coding and the vendor lock 10 What to keep on the shelf

11 COLO, YOUR OWN ROOM, OR RENTED METAL

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

12 READING THE POWER CLAUSE

CUTOVER
5 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Site Part, number 12, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,900/mo	$172/mo	96%	5 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 11 Colo, your own room, or rented metal

12 READING THE POWER CLAUSE

8 STEPS · 5 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 5 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

13 THE CROSS-CONNECT PRICE LIST

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	LOW	30 DAYS

> Synthetic placeholder for the Site Part, number 13, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,370/mo	$412/mo	88%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 12 Reading the power clause

13 THE CROSS-CONNECT PRICE LIST

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

14 TWO FEEDS, ONE OF WHICH YOU HAVE TESTED

CUTOVER
5 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Site Part, number 14, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,820/mo	$88/mo	98%	5 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 07 Optics, coding and the vendor lock 13 The cross-connect price list

14 TWO FEEDS, ONE OF WHICH YOU HAVE TESTED

8 STEPS · 5 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 5 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

15 COOLING, AND THE INLET TEMPERATURE YOU ARE ALLOWED

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	MEDIUM	7 DAYS

> Synthetic placeholder for the Site Part, number 15, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,720/mo	$292/mo	94%	0 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 13 The cross-connect price list 14 Two feeds, one of which you have tested

15 COOLING, AND THE INLET TEMPERATURE YOU ARE ALLOWED

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

16 THE FIRST RACK

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Site Part, number 16, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$580/mo	$88/mo	85%	0 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 08 Tier one, whitebox, or somebody else's three-year-old fleet 13 The cross-connect price list

16 THE FIRST RACK

8 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 0 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

17 WHO GETS IN AT 03:00

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
SITE	**MANAGED SERVICE**	HIGH	30 DAYS

> Synthetic placeholder for the Site Part, number 17, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,360/mo	$304/mo	93%	30 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 11 Colo, your own room, or rented metal 13 The cross-connect price list

17 WHO GETS IN AT 03:00

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

18 THE LOADING DOCK AND THE CUSTOMS FORM

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Site Part, number 18, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,170/mo	$100/mo	98%	20 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 17 Who gets in at 03:00

18 THE LOADING DOCK AND THE CUSTOMS FORM

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

19 CERTIFIED DESTRUCTION

CUTOVER
3 MIN

LAYER	LEAVING	RISK	REVERSIBLE
SITE	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Site Part, number 19, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$670/mo	$268/mo	60%	3 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 11 Colo, your own room, or rented metal 17 Who gets in at 03:00

19 CERTIFIED DESTRUCTION

8 STEPS · 3 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

SITE · 3 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

III

CLUSTER

Everything between a racked machine and a cluster that can take production traffic. Nothing here moves a workload; all of it decides how well the rest of the book goes.

11	7	6	38
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

27

THREE CONTROL-PLANE NODES, THE API VIP AND THE CLUSTER CA

Synthetic placeholder for the Cluster Part, number 27, used only to design the page layouts at realistic length.

20

TALOS, AND THE MACHINE CONFIG

Synthetic placeholder for the Cluster Part, number 20, used only to design the page layouts at realistic length.

21

COLO OR RENTED METAL, AND THE SECOND SITE YOU ARE NOT BUILDING

Synthetic placeholder for the Cluster Part, number 21, used only to design the page layouts at realistic length.

TALOS, AND THE MACHINE CONFIG · COLO OR RENTED METAL, AND THE SECOND SITE YOU ARE NOT BUILDING · THE ADDRESS PLAN, AND THE THREE CLOCKS YOU START TODAY · NETBOOT AND NODE IDENTITY · THE BMC, THE MANAGEMENT VLAN AND THE WAY IN AT 03:00 · TALOS, OR FLATCAR WHEN TALOS WILL NOT DO · NETBOOT, NODE IDENTITY AND FAILURE-DOMAIN LABELS · THREE CONTROL-PLANE NODES, THE API VIP AND THE CLUSTER CA · THE ETCD RESTORE DRILL · CILIUM, WITH KUBE-PROXY OFF · BGP TO THE TOP-OF-RACK, AND THE MTU THAT BREAKS FIVE PER CENT

2 LOW RISK	3 MEDIUM RISK	6 HIGH RISK

20 TALOS, AND THE MACHINE CONFIG

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Cluster Part, number 20, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

CLUSTER

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,440/mo	$304/mo	94%	20 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 14 Two feeds, one of which you have tested 15 Cooling, and the inlet temperature you are allowed 16 The first rack 18 The loading dock and the customs form 19 Certified destruction

20 TALOS, AND THE MACHINE CONFIG

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

21 COLO OR RENTED METAL, AND THE SECOND SITE YOU ARE NOT BUILDING

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	MEDIUM	30 DAYS

> Synthetic placeholder for the Cluster Part, number 21, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,300/mo	$508/mo	61%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

21 COLO OR RENTED METAL, AND THE SECOND SITE YOU ARE NOT BUILDING

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

22 THE ADDRESS PLAN, AND THE THREE CLOCKS YOU START TODAY

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	MEDIUM	IMMEDIATELY

> Synthetic placeholder for the Cluster Part, number 22, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

CLUSTER

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,570/mo	$256/mo	84%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 20 Talos, and the machine config 21 Colo or rented metal, and the second site you are not building

22 THE ADDRESS PLAN, AND THE THREE CLOCKS YOU START TODAY

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

23 NETBOOT AND NODE IDENTITY

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Cluster Part, number 23, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,650/mo	$412/mo	84%	0 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 22 The address plan, and the three clocks you start today

23 NETBOOT AND NODE IDENTITY

8 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

24 THE BMC, THE MANAGEMENT VLAN AND THE WAY IN AT 03:00

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Cluster Part, number 24, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,650/mo	$460/mo	83%	0 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 22 The address plan, and the three clocks you start today

24 THE BMC, THE MANAGEMENT VLAN AND THE WAY IN AT 03:00

8 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

25 TALOS, OR FLATCAR WHEN TALOS WILL NOT DO

CUTOVER
3 MIN

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Cluster Part, number 25, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,350/mo	$460/mo	91%	3 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 22 The address plan, and the three clocks you start today

25 TALOS, OR FLATCAR WHEN TALOS WILL NOT DO

8 STEPS · 3 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 3 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

26 NETBOOT, NODE IDENTITY AND FAILURE-DOMAIN LABELS

CUTOVER
3 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Cluster Part, number 26, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

CLUSTER

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,270/mo	$328/mo	92%	3 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 22 The address plan, and the three clocks you start today

26 NETBOOT, NODE IDENTITY AND FAILURE-DOMAIN LABELS

8 STEPS · 3 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 3 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

27 THREE CONTROL-PLANE NODES, THE API VIP AND THE CLUSTER CA

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	LOW	IMMEDIATELY

> Synthetic placeholder for the Cluster Part, number 27, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

CLUSTER

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,390/mo	$148/mo	89%	0 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 22 The address plan, and the three clocks you start today

27 THREE CONTROL-PLANE NODES, THE API VIP AND THE CLUSTER CA

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

28 THE ETCD RESTORE DRILL

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	LOW	IMMEDIATELY

> Synthetic placeholder for the Cluster Part, number 28, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,190/mo	$484/mo	85%	0 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 20 Talos, and the machine config 26 Netboot, node identity and failure-domain labels

28 THE ETCD RESTORE DRILL

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

29 CILIUM, WITH KUBE-PROXY OFF

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	MEDIUM	30 DAYS

> Synthetic placeholder for the Cluster Part, number 29, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$400/mo	$148/mo	63%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 28 The etcd restore drill

29 CILIUM, WITH KUBE-PROXY OFF

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

30 BGP TO THE TOP-OF-RACK, AND THE MTU THAT BREAKS FIVE PER CENT

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
CLUSTER	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Cluster Part, number 30, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,120/mo	$424/mo	62%	12 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 28 The etcd restore drill 29 Cilium, with kube-proxy off

30 BGP TO THE TOP-OF-RACK, AND THE MTU THAT BREAKS FIVE PER CENT

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

CLUSTER · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

IV

PLATFORM

What developers touch. Get this wrong and the migration succeeds while everybody who has to use it quietly hates you.

11	5	6	74
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

33

THE MIRROR, THEN HARBOR AS THE REGISTRY OF RECORD

Synthetic placeholder for the Platform Part, number 33, used only to design the page layouts at realistic length.

36

ARGO CD, AND THE TWO INFRASTRUCTURE-AS-CODE ESTATES

Synthetic placeholder for the Platform Part, number 36, used only to design the page layouts at realistic length.

31

WORKLOAD IDENTITY WITHOUT THE CLOUD'S

Synthetic placeholder for the Platform Part, number 31, used only to design the page layouts at realistic length.

WORKLOAD IDENTITY WITHOUT THE CLOUD'S · SECRETS: EXTERNAL SECRETS AS THE DATED BRIDGE · THE MIRROR, THEN HARBOR AS THE REGISTRY OF RECORD · COSIGN, TRIVY AND AN ADMISSION POLICY · BUILDS ONTO YOUR OWN METAL · ARGO CD, AND THE TWO INFRASTRUCTURE-AS-CODE ESTATES · EAST-WEST: SECURITY GROUPS INTO NETWORKPOLICY · THE FIRST STATELESS SERVICE, END TO END · THE ROOT OF TRUST AFTER MANAGED KMS · THE SERVERLESS ESTATE IN FOUR PILES · ORCHESTRATED WORKFLOWS: THE DATED EXCEPTION

4 LOW RISK	1 MEDIUM RISK	6 HIGH RISK

31 WORKLOAD IDENTITY WITHOUT THE CLOUD'S

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Platform Part, number 31, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,650/mo	$340/mo	87%	0 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 20 Talos, and the machine config 28 The etcd restore drill

31 WORKLOAD IDENTITY WITHOUT THE CLOUD'S

8 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 0 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

32 SECRETS: EXTERNAL SECRETS AS THE DATED BRIDGE

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Platform Part, number 32, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,000/mo	$340/mo	92%	12 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 31 Workload identity without the cloud's

32 SECRETS: EXTERNAL SECRETS AS THE DATED BRIDGE

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

33 THE MIRROR, THEN HARBOR AS THE REGISTRY OF RECORD

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	LOW	IMMEDIATELY

> Synthetic placeholder for the Platform Part, number 33, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,570/mo	$376/mo	76%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 31 Workload identity without the cloud's

33 THE MIRROR, THEN HARBOR AS THE REGISTRY OF RECORD

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

34 COSIGN, TRIVY AND AN ADMISSION POLICY

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	LOW	30 DAYS

> Synthetic placeholder for the Platform Part, number 34, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,820/mo	$76/mo	98%	0 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 20 Talos, and the machine config 28 The etcd restore drill

34 COSIGN, TRIVY AND AN ADMISSION POLICY

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

35 BUILDS ONTO YOUR OWN METAL

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	LOW	7 DAYS

> Synthetic placeholder for the Platform Part, number 35, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,460/mo	$112/mo	97%	0 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 28 The etcd restore drill 29 Cilium, with kube-proxy off

35 BUILDS ONTO YOUR OWN METAL

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

36 ARGO CD, AND THE TWO INFRASTRUCTURE-AS-CODE ESTATES

CUTOVER
5 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	MEDIUM	IMMEDIATELY

> Synthetic placeholder for the Platform Part, number 36, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,350/mo	$196/mo	96%	5 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 31 Workload identity without the cloud's 32 Secrets: External Secrets as the dated bridge

33 The mirror, then Harbor as the registry of record 34 cosign, Trivy and an admission policy 35 Builds onto your own metal

36 ARGO CD, AND THE TWO INFRASTRUCTURE-AS-CODE ESTATES

8 STEPS · 5 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 5 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

37 EAST-WEST: SECURITY GROUPS INTO NETWORKPOLICY

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Platform Part, number 37, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,380/mo	$496/mo	79%	12 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 36 Argo CD, and the two infrastructure-as-code estates

37 EAST-WEST: SECURITY GROUPS INTO NETWORKPOLICY

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

38 THE FIRST STATELESS SERVICE, END TO END

CUTOVER
5 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Platform Part, number 38, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,030/mo	$412/mo	60%	5 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 37 East-west: security groups into NetworkPolicy

38 THE FIRST STATELESS SERVICE, END TO END

8 STEPS · 5 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 5 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

39 THE ROOT OF TRUST AFTER MANAGED KMS

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Platform Part, number 39, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,110/mo	$100/mo	95%	20 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 11 Colo, your own room, or rented metal 24 The BMC, the management VLAN and the way in at 03:00

39 THE ROOT OF TRUST AFTER MANAGED KMS

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

40 THE SERVERLESS ESTATE IN FOUR PILES

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	LOW	30 DAYS

> Synthetic placeholder for the Platform Part, number 40, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,630/mo	$232/mo	95%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 08 Tier one, whitebox, or somebody else's three-year-old fleet 35 Builds onto your own metal 39 The root of trust after managed KMS

40 THE SERVERLESS ESTATE IN FOUR PILES

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

41 ORCHESTRATED WORKFLOWS: THE DATED EXCEPTION

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
PLATFORM	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Platform Part, number 41, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,570/mo	$436/mo	72%	20 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 14 Two feeds, one of which you have tested 40 The serverless estate in four piles

41 ORCHESTRATED WORKFLOWS: THE DATED EXCEPTION

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

PLATFORM · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

V

DATA

The part that can lose a company its data, and therefore the part with the longest overlap windows, the most rehearsal and the fewest one-way steps.

11	4	5	88
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

50

REDIS: WHICH HALF OF IT IS ACTUALLY A CACHE

Synthetic placeholder for the Data Part, number 50, used only to design the page layouts at realistic length.

52

BUCKETS: THE MIRROR, AND THE S3 API YOUR CODE ASSUMES

Synthetic placeholder for the Data Part, number 52, used only to design the page layouts at realistic length.

42

THE LINKS YOU BUILD FIRST: COLO TO CLOUD, COLO TO OFFICE

Synthetic placeholder for the Data Part, number 42, used only to design the page layouts at realistic length.

THE LINKS YOU BUILD FIRST: COLO TO CLOUD, COLO TO OFFICE · THE POSTGRES PRE-FLIGHT · THE VERIFICATION GATE · POSTGRES ONTO YOUR OWN DISKS · POINT-IN-TIME RECOVERY WITH PGBACKREST · MULTI-ZONE REPLACED: QUORUM FAILOVER AND FENCING · AURORA AND ALLOYDB: THE EXIT IS A STREAM · MYSQL, WHEREVER IT IS MANAGED · REDIS: WHICH HALF OF IT IS ACTUALLY A CACHE · THE QUEUES COME HOME, THE FAN-OUT DOES NOT · BUCKETS: THE MIRROR, AND THE S3 API YOUR CODE ASSUMES

1 LOW RISK	5 MEDIUM RISK	5 HIGH RISK

42 THE LINKS YOU BUILD FIRST: COLO TO CLOUD, COLO TO OFFICE

CUTOVER
5 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	MEDIUM	IMMEDIATELY

> Synthetic placeholder for the Data Part, number 42, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,720/mo	$436/mo	91%	5 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 37 East-west: security groups into NetworkPolicy 39 The root of trust after managed KMS

41 Orchestrated workflows: the dated exception

42 THE LINKS YOU BUILD FIRST: COLO TO CLOUD, COLO TO OFFICE

8 STEPS · 5 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 5 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

43 THE POSTGRES PRE-FLIGHT

CUTOVER
3 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	MEDIUM	30 DAYS

> Synthetic placeholder for the Data Part, number 43, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,370/mo	$316/mo	91%	3 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 09 Burn-in before anything trusts it 32 Secrets: External Secrets as the dated bridge

42 The links you build first: colo to cloud, colo to office

43 THE POSTGRES PRE-FLIGHT

8 STEPS · 3 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 3 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

44 THE VERIFICATION GATE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	MEDIUM	7 DAYS

> Synthetic placeholder for the Data Part, number 44, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,460/mo	$448/mo	87%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 43 The Postgres pre-flight

44 THE VERIFICATION GATE

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

45 POSTGRES ONTO YOUR OWN DISKS

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Data Part, number 45, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,910/mo	$184/mo	95%	30 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 12 Reading the power clause 13 The cross-connect price list 44 The verification gate

45 POSTGRES ONTO YOUR OWN DISKS

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

46 POINT-IN-TIME RECOVERY WITH PGBACKREST

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	MEDIUM	7 DAYS

> Synthetic placeholder for the Data Part, number 46, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,480/mo	$436/mo	71%	0 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 42 The links you build first: colo to cloud, colo to office 44 The verification gate

46 POINT-IN-TIME RECOVERY WITH PGBACKREST

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

47 MULTI-ZONE REPLACED: QUORUM FAILOVER AND FENCING

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Data Part, number 47, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$490/mo	$244/mo	50%	20 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 09 Burn-in before anything trusts it 10 What to keep on the shelf 43 The Postgres pre-flight

47 MULTI-ZONE REPLACED: QUORUM FAILOVER AND FENCING

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

48 AURORA AND ALLOYDB: THE EXIT IS A STREAM

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Data Part, number 48, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,360/mo	$496/mo	89%	20 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 09 Burn-in before anything trusts it 35 Builds onto your own metal 43 The Postgres pre-flight

47 Multi-zone replaced: quorum failover and fencing

48 AURORA AND ALLOYDB: THE EXIT IS A STREAM

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

49 MYSQL, WHEREVER IT IS MANAGED

CUTOVER
5 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Data Part, number 49, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,470/mo	$100/mo	96%	5 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 10 What to keep on the shelf 42 The links you build first: colo to cloud, colo to office 47 Multi-zone replaced: quorum failover and fencing 48 Aurora and AlloyDB: the exit is a stream

49 MYSQL, WHEREVER IT IS MANAGED

8 STEPS · 5 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 5 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

50 REDIS: WHICH HALF OF IT IS ACTUALLY A CACHE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	LOW	7 DAYS

Synthetic placeholder for the Data Part, number 50, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,100/mo	$184/mo	94%	0 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 40 The serverless estate in four piles 43 The Postgres pre-flight

50 REDIS: WHICH HALF OF IT IS ACTUALLY A CACHE

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

51 THE QUEUES COME HOME, THE FAN-OUT DOES NOT

CUTOVER
5 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Data Part, number 51, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,910/mo	$508/mo	87%	5 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 03 Memory is what runs out first 42 The links you build first: colo to cloud, colo to office 43 The Postgres pre-flight 47 Multi-zone replaced: quorum failover and fencing 50 Redis: which half of it is actually a cache

51 THE QUEUES COME HOME, THE FAN-OUT DOES NOT

8 STEPS · 5 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 5 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

52 BUCKETS: THE MIRROR, AND THE S3 API YOUR CODE ASSUMES

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
DATA	MANAGED SERVICE	MEDIUM	30 DAYS

> Synthetic placeholder for the Data Part, number 52, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,990/mo	$100/mo	98%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 43 The Postgres pre-flight 49 MySQL, wherever it is managed 51 The queues come home, the fan-out does not

52 BUCKETS: THE MIRROR, AND THE S3 API YOUR CODE ASSUMES

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

DATA · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

VI
EDGE

How traffic reaches you once the addresses are yours. Also the Part with the most honest advice to keep paying somebody else.

11	4	7	144
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

56

THE FIRST VIP: L2, AND FAILOVER IPS WHEN NOBODY WILL PEER

Synthetic placeholder for the Edge Part, number 56, used only to design the page layouts at realistic length.

62

THE SHADOW EDGE: TWO GATEWAYS AND A WEIGHTED RECORD

Synthetic placeholder for the Edge Part, number 62, used only to design the page layouts at realistic length.

53

YOUR OWN ASN, A /24 AND THE IPV6 YOU ALREADY HAVE

Synthetic placeholder for the Edge Part, number 53, used only to design the page layouts at realistic length.

YOUR OWN ASN, A /24 AND THE IPV6 YOU ALREADY HAVE · TRANSIT: TWO UPSTREAMS AND THE 95TH PERCENTILE · EGRESS OFF THE MANAGED NAT, AND THE ALLOWLISTS YOU RENEGOTIATE · THE FIRST VIP: L2, AND FAILOVER IPS WHEN NOBODY WILL PEER · VIPS OVER BGP: ECMP, BFD AND THE REHASH · TLS OFF THE MANAGED CERTIFICATE SERVICE · ENVOY GATEWAY, AFTER INGRESS-NGINX · BALANCER RULES AND NGINX ANNOTATIONS INTO HTTPROUTE · EDGE AUTHENTICATION AFTER THE BALANCER DID IT · THE SHADOW EDGE: TWO GATEWAYS AND A WEIGHTED RECORD · GO-LIVE: THE SOAK, THE FREEZE, AND HOW YOU ABORT

3 LOW RISK	1 MEDIUM RISK	7 HIGH RISK

53 YOUR OWN ASN, A /24 AND THE IPV6 YOU ALREADY HAVE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	MEDIUM	7 DAYS

> Synthetic placeholder for the Edge Part, number 53, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,100/mo	$172/mo	94%	0 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 26 Netboot, node identity and failure-domain labels 39 The root of trust after managed KMS 45 Postgres onto your own disks

53 YOUR OWN ASN, A /24 AND THE IPV6 YOU ALREADY HAVE

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

54 TRANSIT: TWO UPSTREAMS AND THE 95TH PERCENTILE

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Edge Part, number 54, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,990/mo	$340/mo	93%	12 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 49 MySQL, wherever it is managed 53 Your own ASN, a /24 and the IPv6 you already have

54 TRANSIT: TWO UPSTREAMS AND THE 95TH PERCENTILE

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

55 EGRESS OFF THE MANAGED NAT, AND THE ALLOWLISTS YOU RENEGOTIATE

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Edge Part, number 55, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,540/mo	$160/mo	96%	20 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 43 The Postgres pre-flight 53 Your own ASN, a /24 and the IPv6 you already have

55 EGRESS OFF THE MANAGED NAT, AND THE ALLOWLISTS YOU RENEGOTIATE

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

56 THE FIRST VIP: L2, AND FAILOVER IPS WHEN NOBODY WILL PEER

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	LOW	IMMEDIATELY

> Synthetic placeholder for the Edge Part, number 56, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,210/mo	$484/mo	60%	0 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 55 Egress off the managed NAT, and the allowlists you renegotiate

56 THE FIRST VIP: L2, AND FAILOVER IPS WHEN NOBODY WILL PEER

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

☼ EDGE · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

57 VIPS OVER BGP: ECMP, BFD AND THE REHASH

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Edge Part, number 57, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,180/mo	$304/mo	93%	20 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 48 Aurora and AlloyDB: the exit is a stream 56 The first VIP: L2, and failover IPs when nobody will peer

57 VIPS OVER BGP: ECMP, BFD AND THE REHASH

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

58 TLS OFF THE MANAGED CERTIFICATE SERVICE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	LOW	IMMEDIATELY

> Synthetic placeholder for the Edge Part, number 58, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$400/mo	$112/mo	72%	0 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 47 Multi-zone replaced: quorum failover and fencing 49 MySQL, wherever it is managed

54 Transit: two upstreams and the 95th percentile 55 Egress off the managed NAT, and the allowlists you renegotiate

58 TLS OFF THE MANAGED CERTIFICATE SERVICE

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

☼ EDGE · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

59 ENVOY GATEWAY, AFTER INGRESS-NGINX

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Edge Part, number 59, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,350/mo	$184/mo	97%	30 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 57 VIPs over BGP: ECMP, BFD and the rehash 58 TLS off the managed certificate service

59 ENVOY GATEWAY, AFTER INGRESS-NGINX

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

☼ EDGE · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

60 BALANCER RULES AND NGINX ANNOTATIONS INTO HTTPROUTE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	LOW	30 DAYS

> Synthetic placeholder for the Edge Part, number 60, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,570/mo	$256/mo	84%	0 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 44 The verification gate 46 Point-in-time recovery with pgBackRest

60 BALANCER RULES AND NGINX ANNOTATIONS INTO HTTPROUTE

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

☼ EDGE · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

61 EDGE AUTHENTICATION AFTER THE BALANCER DID IT

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	30 DAYS

> Synthetic placeholder for the Edge Part, number 61, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,840/mo	$448/mo	76%	30 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 55 Egress off the managed NAT, and the allowlists you renegotiate 60 Balancer rules and nginx annotations into HTTPRoute

61 EDGE AUTHENTICATION AFTER THE BALANCER DID IT

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

62 THE SHADOW EDGE: TWO GATEWAYS AND A WEIGHTED RECORD

CUTOVER
12 MIN

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Edge Part, number 62, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,620/mo	$304/mo	95%	12 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 49 MySQL, wherever it is managed 54 Transit: two upstreams and the 95th percentile

62 THE SHADOW EDGE: TWO GATEWAYS AND A WEIGHTED RECORD

8 STEPS · 12 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 12 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

63 GO-LIVE: THE SOAK, THE FREEZE, AND HOW YOU ABORT

CUTOVER
20 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
EDGE	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Edge Part, number 63, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$3,460/mo	$148/mo	96%	20 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 49 MySQL, wherever it is managed 54 Transit: two upstreams and the 95th percentile

62 The shadow edge: two gateways and a weighted record

63 GO-LIVE: THE SOAK, THE FREEZE, AND HOW YOU ABORT

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

EDGE

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

EDGE · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

VII

WATCH

Running it once it is yours. Every repatriation article stops before this Part, and every repatriation that regrets itself failed inside it.

11	6	6	103
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

65

THE FLEET ROLL WITH NO THIRTEENTH MACHINE

Synthetic placeholder for the Watch Part, number 65, used only to design the page layouts at realistic length.

67

SPARES, RMA AND WHICH RACK UNIT

Synthetic placeholder for the Watch Part, number 67, used only to design the page layouts at realistic length.

64

THE UPGRADE CALENDAR AND THE CONTROL-PLANE UPGRADE

Synthetic placeholder for the Watch Part, number 64, used only to design the page layouts at realistic length.

THE UPGRADE CALENDAR AND THE CONTROL-PLANE UPGRADE · THE FLEET ROLL WITH NO THIRTEENTH MACHINE · THE 03:00 DISK · SPARES, RMA AND WHICH RACK UNIT · CAPACITY PLANNING, AND THE CLOUD FOOTPRINT YOU KEEP · VELERO, THE OFF-SITE COPY AND THE REBUILD DRILL · THE ROTA · ALERTS THAT SURVIVE THE CLUSTER, AND THE PAGER YOU RENT · THE PATCH SLA AFTER NVD STOPPED · AUDIT LOGS AFTER THE PROVIDER'S · THE AUDITOR'S CHECKLIST, AND THE CONTRACTS NOBODY READ

3 LOW RISK	2 MEDIUM RISK	6 HIGH RISK

64 THE UPGRADE CALENDAR AND THE CONTROL-PLANE UPGRADE

CUTOVER
30 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Watch Part, number 64, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,810/mo	$172/mo	96%	30 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 56 The first VIP: L2, and failover IPs when nobody will peer 58 TLS off the managed certificate service

59 Envoy Gateway, after ingress-nginx

64 THE UPGRADE CALENDAR AND THE CONTROL-PLANE UPGRADE

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

65 THE FLEET ROLL WITH NO THIRTEENTH MACHINE

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	LOW	7 DAYS

> Synthetic placeholder for the Watch Part, number 65, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,210/mo	$400/mo	67%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 56 The first VIP: L2, and failover IPs when nobody will peer 58 TLS off the managed certificate service

64 The upgrade calendar and the control-plane upgrade

65 THE FLEET ROLL WITH NO THIRTEENTH MACHINE

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH

WATCH · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

66 THE 03:00 DISK

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	LOW	30 DAYS

> Synthetic placeholder for the Watch Part, number 66, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,270/mo	$436/mo	90%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 65 The fleet roll with no thirteenth machine

66 THE 03:00 DISK

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

67 SPARES, RMA AND WHICH RACK UNIT

CUTOVER
30 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	HIGH	IMMEDIATELY

> Synthetic placeholder for the Watch Part, number 67, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$5,440/mo	$460/mo	92%	30 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 53 Your own ASN, a /24 and the IPv6 you already have 55 Egress off the managed NAT, and the allowlists you renegotiate

56 The first VIP: L2, and failover IPs when nobody will peer 58 TLS off the managed certificate service 59 Envoy Gateway, after ingress-nginx

60 Balancer rules and nginx annotations into HTTPRoute 62 The shadow edge: two gateways and a weighted record

63 Go-live: the soak, the freeze, and how you abort 65 The fleet roll with no thirteenth machine

67 SPARES, RMA AND WHICH RACK UNIT

8 STEPS · 30 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 30 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

68 CAPACITY PLANNING, AND THE CLOUD FOOTPRINT YOU KEEP

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	LOW	7 DAYS

> Synthetic placeholder for the Watch Part, number 68, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,930/mo	$64/mo	97%	0 min	5 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 60 Balancer rules and nginx annotations into HTTPRoute 67 Spares, RMA and which rack unit

68 CAPACITY PLANNING, AND THE CLOUD FOOTPRINT YOU KEEP

8 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 0 MIN · LOW RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

69 VELERO, THE OFF-SITE COPY AND THE REBUILD DRILL

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	MEDIUM	IMMEDIATELY

> Synthetic placeholder for the Watch Part, number 69, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,720/mo	$88/mo	98%	0 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 05 Network cards, and the two ports you actually need 47 Multi-zone replaced: quorum failover and fencing

67 Spares, RMA and which rack unit

69 VELERO, THE OFF-SITE COPY AND THE REBUILD DRILL

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH

WATCH · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

70 THE ROTA

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Watch Part, number 70, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,360/mo	$244/mo	94%	20 min	7 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 47 Multi-zone replaced: quorum failover and fencing 49 MySQL, wherever it is managed

55 Egress off the managed NAT, and the allowlists you renegotiate 60 Balancer rules and nginx annotations into HTTPRoute

69 Velero, the off-site copy and the rebuild drill

70 THE ROTA

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

71 ALERTS THAT SURVIVE THE CLUSTER, AND THE PAGER YOU RENT

CUTOVER
20 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	HIGH	7 DAYS

> Synthetic placeholder for the Watch Part, number 71, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to `1` and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$4,360/mo	$436/mo	90%	20 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 51 The queues come home, the fan-out does not 53 Your own ASN, a /24 and the IPv6 you already have

71 ALERTS THAT SURVIVE THE CLUSTER, AND THE PAGER YOU RENT

8 STEPS · 20 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 20 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

72 THE PATCH SLA AFTER NVD STOPPED

CUTOVER
3 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Watch Part, number 72, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,120/mo	$352/mo	69%	3 min	1 day	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 51 The queues come home, the fan-out does not 53 Your own ASN, a /24 and the IPv6 you already have

71 Alerts that survive the cluster, and the pager you rent

72 THE PATCH SLA AFTER NVD STOPPED

8 STEPS · 3 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH

WATCH · 3 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

73 AUDIT LOGS AFTER THE PROVIDER'S

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	HIGH	NO

> Synthetic placeholder for the Watch Part, number 73, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$2,200/mo	$88/mo	96%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 47 Multi-zone replaced: quorum failover and fencing 48 Aurora and AlloyDB: the exit is a stream

53 Your own ASN, a /24 and the IPv6 you already have 56 The first VIP: L2, and failover IPs when nobody will peer

58 TLS off the managed certificate service 60 Balancer rules and nginx annotations into HTTPRoute

62 The shadow edge: two gateways and a weighted record 63 Go-live: the soak, the freeze, and how you abort

71 Alerts that survive the cluster, and the pager you rent 72 The patch SLA after NVD stopped

73 AUDIT LOGS AFTER THE PROVIDER'S

8 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. This Move is irreversible: there is no way back. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 0 MIN · HIGH RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

74 THE AUDITOR'S CHECKLIST, AND THE CONTRACTS NOBODY READ

CUTOVER
0 MIN

LAYER	LEAVING	RISK	REVERSIBLE
WATCH	MANAGED SERVICE	MEDIUM	IMMEDIATELY

> Synthetic placeholder for the Watch Part, number 74, used only to design the page layouts at realistic length.

AWS	**RDS for PostgreSQL**	set the `rds.logical_replication` parameter to 1 and reboot; Aurora needs the same flag but replicates from the writer endpoint only.
GOOGLE	**Cloud SQL for PostgreSQL**	set the `cloudsql.logical_decoding` flag, which also reboots, and grant the `cloudsqlexternalsync` role before creating a publication.
AZURE	**Database for PostgreSQL Flexible Server**	set `wal_level` to `logical` in server parameters; Single Server has no logical decoding at all and has to be dumped and restored instead.

BEFORE YOU START

ACCESS

- Superuser or the provider's nearest equivalent on the source, with logical decoding already enabled and the reboot taken
- A maintenance window agreed with whoever owns the application

SOFTWARE

- CloudNativePG 1.25 or newer, installed and running a test cluster you have already failed over
- `psql` and `pg_dump` at a version at least as new as the server
- `pgbackrest` 2.54 or newer, writing to a repository in a third location

HARDWARE

- Local NVMe on at least three nodes, with a storage class that pins a volume to its node
- Enough disk for the database plus fifty per cent, measured rather than estimated

WHY THIS WORKS

Every managed PostgreSQL is PostgreSQL with a control plane bolted on, and the PostgreSQL underneath speaks the same replication protocol as any other instance. A publication on the source and a subscription on the destination move the data continuously while the application keeps writing, which turns a migration into a wait. What remains at cutover is draining the last transactions, promoting the new primary and pointing the application at it. The twelve minutes are not the copy; they are the verification you should refuse to skip.

SWAP

For a database under 50 GB with a tolerant maintenance window, `pg_dump` and restore is simpler and finishes inside an hour. Logical replication earns its complexity above roughly that size.

DO IT FASTER

Run the initial sync from a read replica rather than the primary. It costs an extra instance for a day and takes the load off the database serving traffic.

WATCH OUT

Logical replication does not carry sequences, large objects, or DDL. Every one of those has ended a cutover that otherwise went perfectly.

LEFTOVERS

The publication and subscription stay in place until you drop them. Drop them deliberately after the thirty days, or the source keeps retaining write-ahead log for a subscriber nobody is reading.

$1,570/mo	$268/mo	83%	0 min	3 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

NEEDS 49 MySQL, wherever it is managed 54 Transit: two upstreams and the 95th percentile

55 Egress off the managed NAT, and the allowlists you renegotiate 72 The patch SLA after NVD stopped 73 Audit logs after the provider's

74 THE AUDITOR'S CHECKLIST, AND THE CONTRACTS NOBODY READ

8 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Take a `pgbackrest` backup of the destination's empty cluster and restore it into a scratch namespace. A backup nobody restored is not a backup, and this is the cheapest moment to find that out.

02 Copy the schema only, with `pg_dump --schema-only`, and apply it to the destination. Create every table but no indexes on large tables yet; they slow the initial sync and are faster to build afterwards.

03 Create a publication on the source for all tables, and a subscription on the destination. Watch `pg_stat_subscription` until the initial copy finishes and the lag settles under a second.

04 Build the deferred indexes on the destination, then compare row counts table by table and checksum a sample of the largest three. Do not proceed on a count that disagrees, however small the difference looks.

05 Open the maintenance window. Stop the application's writers, wait for replication lag to reach zero, and confirm it with `pg_current_wal_lsn` on both sides.

06 Advance every sequence on the destination past its source value. A migration that forgets the sequences works perfectly until the first insert collides, which is usually about ninety seconds later.

07 Repoint the application's connection string at the destination and start the writers. Watch error rates and query latency for ten minutes before you call it done.

08 Leave the managed instance running, and leave its automated backups on, for the full thirty days.

ROLLBACK

Until step seven the source is still the primary and still authoritative, so backing out is stopping the subscription and doing nothing else. The point of no return is the first write that lands on the destination: from that moment the two databases have diverged and returning means replicating backwards, not switching back. If you must return after that, stop the writers, set up the reverse subscription from destination to source, wait for it to drain, and repoint. Keep the managed instance and its automated backups for thirty days so a restore to a known good point stays possible.

WATCH · 0 MIN · MEDIUM RISK

The managed instance, its read replicas and its snapshots — after thirty days, and after a full billing cycle has shown the line gone.

A FEW THINGS WORTH KNOWING

None of these is a Move. They are the ten things that make the other 74 work, and most of them are the things people learn in the second year rather than the first.

THE BILL LAGS THE CHANGE BY A MONTH

A resource stopped on the third still bills for the first two days, and a reservation you no longer use bills for a year. Judge a Move by the invoice after next, never by the console.

CAPACITY YOU OWN IS CAPACITY YOU HAVE

The cloud taught a generation to treat headroom as waste. On your own metal, idle capacity is the thing that absorbs a bad deploy at four in the afternoon. Buy thirty per cent more than the model says.

TWO PEOPLE KNOW, OR NOBODY DOES

The single most common failure mode of a self-run platform is not hardware. It is one engineer who understands the network, and a holiday.

HARDWARE FAILS PREDICTABLY AND BORINGLY

Disks, fans, PSUs, optics, in that order, at a rate you can put in a spreadsheet. It is not the drama people expect. The drama is in the change you made on Friday.

KEEP ONE FOOT IN THE CLOUD ON PURPOSE

An account with a working credential, a Terraform state and a warm standby is not a failure of nerve. It is the cheapest disaster recovery site you will ever have.

THE SAVING IS REAL; THE FREEDOM IS THE POINT

Most teams start this for the bill and finish it for the other thing: being able to answer a question about your own system without opening a support case.

WRITE THE RUNBOOK BEFORE YOU NEED IT

Every Move in this book is a runbook because that is the artefact that survives the person who wrote it. A migration that leaves no runbooks behind has to be done again by whoever comes next.

YOU CAN GO BACK

The industry talks about repatriation as though it were one-way. It is not. 16 of the 74 Moves here are genuinely irreversible and the other 58 are not, and knowing which is which is most of the skill.

The best infrastructure decision you make this year will probably be unglamorous, take a fortnight, save a five-figure sum and never be written up anywhere. That is rather the point.

THE BOOK

74 Moves across five Parts. 33 cost no downtime at all, 16 cannot be undone, and the whole book executed end to end costs 641 minutes of user-visible outage.

THE TYPE

Set in Archivo, drawn by Omnibus-Type, and JetBrains Mono, drawn by Philipp Nurullin and Konstantin Bulenkov. Both are open source. Every number in this book is set in the mono.

THE NUMBERS

Every price is a public list rate observed while writing, before any discount, and every saving is illustrative of a shape rather than a quotation. Check the arithmetic against your own invoice.

Disclosure: the author founded OneUptime, which this book recommends for alerting, on-call, incidents and status pages. It is recommended because it is Apache-2.0 licensed, self-hostable, and does in one place what that Part would otherwise ask you to assemble from four separate tools. The alternatives named beside it are real ones, and a reader who prefers them loses nothing else in this book.

HackerBay.io · Version 0.1.0 · The Moves and the typesetter that made this book are open source at github.com/hackerbay/back-to-metal